This starts the server, although the HTTP constructor requires an implicit `ActorSystem`. The server is started with `newServerAt` on the host and port of your choice, and then you can `bind` a `Route` to that server. Note that the binding to the server is wrapped in a future, which means it will be created eventually. That takes time—only a little, but time is part of the equation.

It is recommended to provide a way to shut down the server gracefully. To do this, you can include a listener that registers each new line in the sbt shell so that when Enter is pressed, the shutdown is triggered by unbinding the server and exiting the `ActorSystem`:

```
object HttpServer {
  ...

    StdIn.readLine()                          ❶
    bindingFuture
      .flatMap(_.unbind())                    ❷
      .onComplete(_ => system.terminate())    ❸
  ...
}
```

❶ Listens to the input in the sbt console

❷ Unbinds the server

❸ When unbinding is complete, terminates the system

Just as the HTTP server requires an implicit `ActorSystem`, `bindingFuture` requires an execution context to run. This is available in the `ActorSystem`, and you can use it for now, although you'll learn better options:

```
implicit val executionContext = system.executionContext
```

All of this is put together in the following listing.